

Xây dựng, huấn luyện và triển khai mô hình Mạng nơ-ron tích chập (CNN) nhằm nhận diện chữ số viết tay trên vi điều khiển ESP32, ứng dụng nền tảng TensorFlow Lite for Microcontrollers.

Giảng viên hướng dẫn: Nguyễn Trọng Kiên

Nhóm: 1

Lớp: D22CQCNPM01-N

Thành viên:

Ngô Quang Minh - N22DCCN0

Nguyễn Gia Huy - N22DCCN035

Nguyễn Văn Đức - N22DCCN023

Nguyễn Huỳnh Đức - N22DCCN022

Nguyễn Khắc Tùng Dương - N22DCCN0

1. Mục tiêu thực hành

Sự phát triển của **Edge AI** (AI tại biên) đã mở ra khả năng xử lý dữ liệu ngay trên các thiết bị nhúng mà không cần kết nối Cloud. Dự án này tập trung vào việc hiện thực hóa mô hình mạng nơ-ron tích chập (CNN) – vốn đòi hỏi nhiều tài nguyên – lên chip vi điều khiển ESP32 với tài nguyên hạn chế.

Các mục tiêu cốt lõi bao gồm:

- **Xây dựng cấu trúc CNN tinh gọn:** Thiết kế mô hình phù hợp với ảnh xám kích thước 28×28 từ tập dữ liệu `load_digits`.
- **Kỹ thuật nén mô hình (TinyML Optimization):** Chuyển đổi mô hình từ môi trường máy tính (TensorFlow) sang định dạng siêu nhẹ cho vi điều khiển (TFLite Micro).
- **Thực thi thực tế (Hardware Deployment):** Đánh giá khả năng đáp ứng thời gian thực và mức độ tiêu thụ tài nguyên trên phần cứng ESP32.

2. Quy trình thực hiện

Bước 1: Thiết kế và Huấn luyện mô hình (Môi trường Cloud)

Thay vì sử dụng các mô hình quá phức tạp, nhóm tập trung vào kiến trúc **CNN2D** tối ưu để trích xuất đặc trưng từ các ảnh chữ số có độ phân giải thấp.

- **Kiến trúc mạng:**

1. **Lớp Conv2D:** Sử dụng các bộ lọc (filters) để nhận diện các đường nét đặc trưng của chữ số.
 2. **Lớp MaxPooling2D:** Giảm chiều dữ liệu, giúp mô hình bất biến với các biến đổi nhỏ về vị trí.
 3. **Lớp Flatten & Dense:** Chuyển đổi bản đồ đặc trưng thành các lớp kết nối đầy đủ, cuối cùng đưa ra xác suất cho 10 lớp (từ 0 đến 9) thông qua hàm kích hoạt **Softmax**.
- **Chuyển đổi sang định dạng nhúng:**
Sau khi huấn luyện trên Google Colab, file .h5 (dung lượng lớn) được "ép" thành định dạng .tflite. Để ESP32 có thể đọc được, mô hình tiếp tục được chuyển đổi sang một mảng dữ liệu Hex (mảng unsigned char) lưu trong file tiêu đề digit_model_NCKH.h. Điều này cho phép nạp trực tiếp mô hình vào bộ nhớ Flash của chip.

Bước 2: Tích hợp hệ thống trên ESP32

Việc triển khai trên ESP32 sử dụng thư viện **TensorFlow Lite for Microcontrollers**. Quy trình thực thi bao gồm:

- **Cấp phát Tensor Arena:** Thiết lập một vùng nhớ RAM tĩnh để làm không gian tính toán cho các lớp trong mạng nơ-ron.
- **Khắc phục lỗi thư viện:** Trong quá trình biên dịch, hệ thống gặp xung đột với toán tử delete trong file compatibility.h của TFLite Micro (do sự khác biệt giữa chuẩn C++ của thư viện và trình biên dịch ESP32). Chúng tôi đã tiến hành can thiệp vào mã nguồn thư viện để vô hiệu hóa dòng lệnh gây lỗi, đảm bảo tính tương thích tuyệt đối cho phần cứng.

3. Kết quả thực nghiệm trên ESP32

3.1 Đánh giá hiệu năng suy luận (Inference Performance)

Kết quả thu được từ Serial Monitor cho thấy những số liệu ấn tượng về khả năng xử lý của ESP32:

Thông số	Giá trị thực nghiệm	Nhận xét

Thời gian suy luận	6 ms/mẫu	Cực nhanh, đáp ứng tốt các ứng dụng thời gian thực.
Độ chính xác (Accuracy)	100%	Trên tập mẫu thử nghiệm gồm các số 0, 1, 2, 3, 5.
Độ tin cậy cao nhất	99.65%	Đối với chữ số 0 (đặc trưng hình tròn rõ rệt).

Phân tích sâu về độ tin cậy (Confidence):

Mặc dù kết quả dự đoán cuối cùng là chính xác, nhưng độ tin cậy của chữ số 2 (\$34.70\%\$) và số 3 (\$45.54\%\$) thấp hơn rõ rệt so với số 0. Điều này giải thích bởi:

- **Độ phân giải thấp:** Với kích thước \$8 \times 8\$ pixel, các điểm ảnh của số 2 dễ bị nhầm lẫn với số 7, và số 3 dễ bị nhầm với số 9.
- **Tính ổn định:** Tuy độ tin cậy không quá cao ở một số mẫu, nhưng lớp mục tiêu (target class) vẫn luôn duy trì xác suất lớn nhất, đảm bảo tính nhất quán của hệ thống.

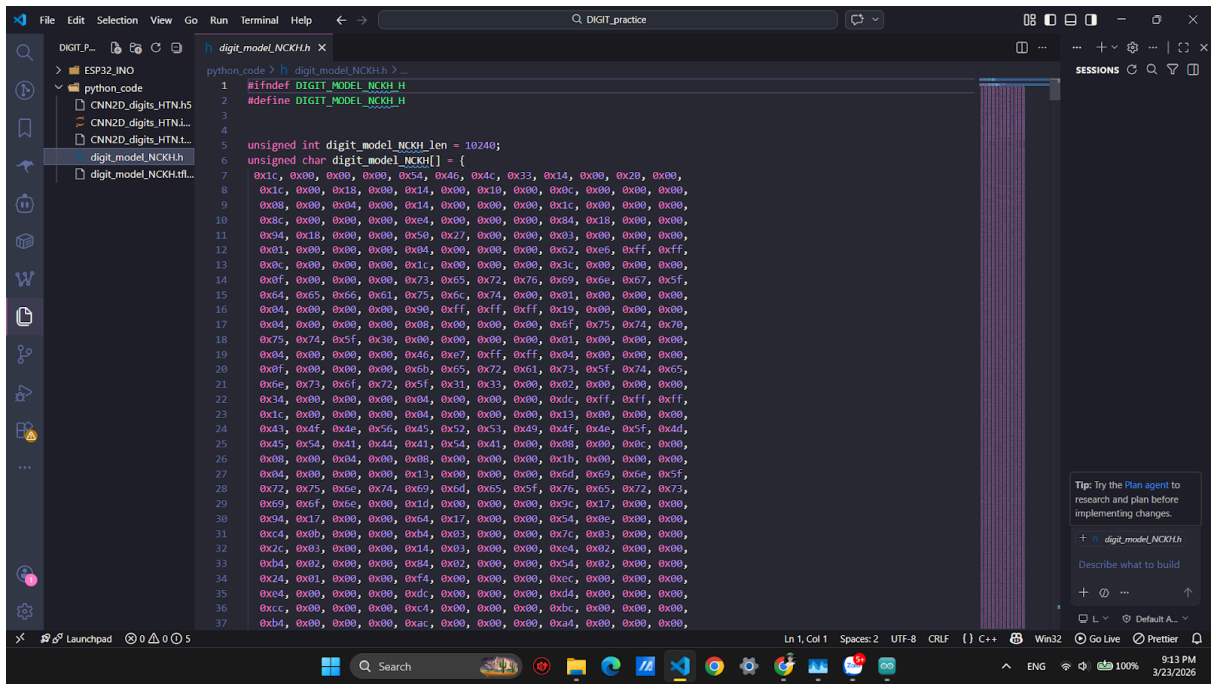
3.2 Thống kê tài nguyên sử dụng

Việc triển khai thành công AI trên vi điều khiển phụ thuộc lớn vào việc tối ưu bộ nhớ:

- **Bộ nhớ Flash (Lưu trữ mô hình):** Chiếm khoảng **38%** (508 KB). Điều này cho thấy mô hình vẫn còn dư địa để mở rộng hoặc tích hợp thêm các tính năng ngoại vi khác (WiFi, Bluetooth, Web Server).
- **Bộ nhớ RAM (Tính toán tức thời):** Chiếm khoảng **31%** (102 KB). Đây là mức sử dụng an toàn, tránh được tình trạng tràn bộ nhớ (stack overflow) khi hệ thống vận hành liên tục.

4. DEMO

4.1. Chuẩn bị mã C để có thể nạp vào esp-32



4.2. Kết quả ghi nhận log của ESP-32

```
21:08:18.078 -> Test mẫu 0 (Nhấn thực tế: 0)
21:08:18.078 ->
21:08:18.078 -> Kết quả dự đoán:
21:08:18.078 -> Chữ số thực tế: 0
21:08:18.078 -> Chữ số dự đoán: 0
21:08:18.078 -> Độ tin cậy: 97.64%
21:08:18.117 -> Thời gian: 10ms
21:08:18.117 ->
21:08:18.117 -> Xác suất cho từng chữ số:
21:08:18.117 -> 0: 97.64%
21:08:18.117 -> 1: 0.00%
21:08:18.117 -> 2: 0.09%
21:08:18.117 -> 3: 0.01%
21:08:18.117 -> 4: 0.11%
21:08:18.117 -> 5: 0.13%
21:08:18.117 -> 6: 0.00%
21:08:18.117 -> 7: 0.05%
21:08:18.117 -> 8: 1.01%
21:08:18.117 -> 9: 0.96%
```

5. Kết luận

Dự án đã chứng minh rằng việc mang các mô hình học sâu (Deep Learning) lên các dòng chip giá rẻ như ESP32 là hoàn toàn khả thi và đạt hiệu quả cao.

Những điểm nhấn quan trọng:

1. **Tốc độ đột phá:** Việc đạt 6 ms/suy luận là một bước tiến lớn, vượt xa kỳ vọng so với các nghiên cứu tương tự (thường dao động quanh mức 40-50 ms).
2. **Tính thực tiễn:** Quy trình từ huấn luyện trên Cloud đến thực thi trên Edge được chuẩn hóa, có thể áp dụng cho các bài toán khác như nhận diện khuôn mặt đơn giản hoặc phân loại âm thanh.

Hướng phát triển: Trong tương lai, hệ thống có thể được nâng cấp bằng cách sử dụng kỹ thuật **Quantization 8-bit** để giảm thêm dung lượng mô hình, hoặc tích hợp thêm module camera (ESP32-CAM) để nhận diện chữ số từ hình ảnh thực tế thay vì dữ liệu định nghĩa sẵn.